

UNIVERSITY OF ASIA PACIFIC

DEPARTMENT OF CSE

Lab Report – 07

Page Replacement Algorithms (FIFO & MRU)

COURSE CODE: CSE 402
COURSE TITLE: Operating System Lab

SUBMITTED BY:

NAME	: Swagoto Utsab Singha Roy
STUDENT ID	: 23101124
SEMESTER	: 4th YEAR 1st SEMESTER
SECTION	: C–1
DATE	: 19 – 10 – 2026

SUBMITTED TO:

ATIA RAHMAN ORTHI
LECTURER
DEPARTMENT OF CSE, UAP

1. Problem Statement

The objective of this lab is to implement the FIFO (First In First Out) and MRU (Most Recently Used) page replacement algorithms in Python. Both algorithms are tested on the same reference string with the same number of frames. Their results are then compared to find out which one gives fewer page faults and why.

Parameter	Value
Reference String	1, 2, 3, 4, 5, 1, 2, 3, 1
Number of Frames	3
Total References	9

For each page in the reference string, the program prints the current frames and whether the page was a hit or a fault. After the whole string is processed it prints total hits, total faults, hit ratio, fault ratio, the insert sequence and the replace sequence. In the replace sequence each entry shows the removed page first and then the new page.

FIFO (First In First Out)

In FIFO the frames work like a queue. If a page fault occurs and all frames are already full, the page which entered memory first is removed and the new page is added at the end. When a hit happens nothing changes in the frames. The algorithm is simple to implement, but it does not consider whether a page is being used often or not. So sometimes it removes a page that will be needed again very soon.

MRU (Most Recently Used)

MRU removes the page that was used most recently when a replacement is needed. For this, every time a hit occurs the page is moved to the end of the list, so the last element is always the most recently used page. In the code this is done with `remove()` and `append()`, and `pop()` removes the last element during replacement. MRU is useful when the same set of pages is accessed in a loop and the loop is bigger than the number of frames.

2. Source Code

2.1 FIFO

```python
def fifo(pages, frame_size):
    frames = []
    hits = 0
    faults = 0
    insert_seq = []
    replace_seq = []
    print(f"Reference String : {pages}")
    print(f"Frame Size : {frame_size}")
    print(f"Page | Frames | Result")
    for page in pages:
        if page in frames:
            hits += 1
            result = "HIT"
        else:
            faults += 1
            result = "FAULT"
            insert_seq.append(page)

            if len(frames) < frame_size:
                frames.append(page)
            else:
                removed = frames.pop(0)
                replace_seq.append(f"{removed} {page}")
                frames.append(page)
        print(f" {page} | {str(frames):<10} | {result}")
    total = len(pages)
    print(f"Hits : {hits}")
    print(f"Faults : {faults}")
    print(f"Hit Ratio : {hits}/{total} = {hits/total:.2f}")
    print(f"Fault Ratio : {faults}/{total} = {faults/total:.2f}")
    print(f"Insert seq : {insert_seq}")
    print(f"Replace seq : {replace_seq}")
```

```python
pages = [1, 2, 3, 4, 5, 1, 2, 3, 1]
frame_size = 3
fifo(pages, frame_size)
```

Listing 1: FIFO Page Replacement (`LAB-07-FIFO_Page_Replacement_Algo.py`)

2.2 MRU

```python
def mru(pages, frame_size):
    frames = []
    hits = 0
    faults = 0
    insert_seq = []
    replace_seq = []
    print(f"Reference String : {pages}")
    print(f"Frame Size : {frame_size}")
    print(f"Page | Frames | Result")
    for page in pages:
        if page in frames:
            hits += 1
            result = "HIT"
            frames.remove(page)
            frames.append(page)
        else:
            faults += 1
            result = "FAULT"
            insert_seq.append(page)

            if len(frames) < frame_size:
                frames.append(page)
            else:
                removed = frames.pop()
                replace_seq.append(f"{removed} {page}")
                frames.append(page)
        print(f" {page} | {str(frames):<10} | {result}")
    total = len(pages)
    print(f"Hits : {hits}")
    print(f"Faults : {faults}")
    print(f"Hit Ratio : {hits}/{total} = {hits/total:.2f}")
    print(f"Fault Ratio : {faults}/{total} = {faults/total:.2f}")
    print(f"Insert seq : {insert_seq}")
    print(f"Replace seq : {replace_seq}")
pages = [1, 2, 3, 4, 5, 1, 2, 3, 1]
frame_size = 3
mru(pages, frame_size)
```

Listing 2: MRU Page Replacement (`LAB-08-MRU_Page_Replacement_Algo.py`)

3. Output

3.1 FIFO

```
Reference String : [1, 2, 3, 4, 5, 1, 2, 3, 1]
Frame Size : 3
Page | Frames | Result
 1 | [1]        | FAULT
 2 | [1, 2]     | FAULT
 3 | [1, 2, 3]  | FAULT
 4 | [2, 3, 4]  | FAULT
 5 | [3, 4, 5]  | FAULT
 1 | [4, 5, 1]  | FAULT
 2 | [5, 1, 2]  | FAULT
 3 | [1, 2, 3]  | FAULT
 1 | [1, 2, 3]  | HIT
Hits : 1
Faults : 8
Hit Ratio : 1/9 = 0.11
Fault Ratio : 8/9 = 0.89
Insert seq : [1, 2, 3, 4, 5, 1, 2, 3]
Replace seq : ['1 4', '2 5', '3 1', '4 2', '5 3']
```

Output of ***LAB-07-FIFO_Page_Replacement_Algo.py***

Step	Page	Frames (oldest to newest)	Result
1	1	[1]	FAULT
2	2	[1, 2]	FAULT
3	3	[1, 2, 3]	FAULT
4	4	[2, 3, 4]	FAULT (1 out)
5	5	[3, 4, 5]	FAULT (2 out)
6	1	[4, 5, 1]	FAULT (3 out)
7	2	[5, 1, 2]	FAULT (4 out)
8	3	[1, 2, 3]	FAULT (5 out)
9	1	[1, 2, 3]	HIT

Insert sequence: 1, 2, 3, 4, 5, 1, 2, 3
Replace sequence: $1 \to 4,\ 2 \to 5,\ 3 \to 1,\ 4 \to 2,\ 5 \to 3$
Hits = 1, Faults = 8, Hit ratio = $1/9 = 0.11$, Fault ratio = $8/9 = 0.89$

3.2 MRU

```
Reference String : [1, 2, 3, 4, 5, 1, 2, 3, 1]
Frame Size : 3
Page | Frames | Result
 1 | [1]        | FAULT
 2 | [1, 2]     | FAULT
 3 | [1, 2, 3]  | FAULT
 4 | [1, 2, 4]  | FAULT
 5 | [1, 2, 5]  | FAULT
 1 | [2, 5, 1]  | HIT
 2 | [5, 1, 2]  | HIT
 3 | [5, 1, 3]  | FAULT
 1 | [5, 3, 1]  | HIT
Hits : 3
Faults : 6
Hit Ratio : 3/9 = 0.33
Fault Ratio : 6/9 = 0.67
Insert seq : [1, 2, 3, 4, 5, 3]
Replace seq : ['3 4', '4 5', '2 3']
```

Output of `LAB-08-MRU_Page_Replacement_Algo.py`

Step	Page	Frames (least to most recent)	Result
1	1	[1]	FAULT
2	2	[1, 2]	FAULT
3	3	[1, 2, 3]	FAULT
4	4	[1, 2, 4]	FAULT (3 out)
5	5	[1, 2, 5]	FAULT (4 out)
6	1	[2, 5, 1]	HIT
7	2	[5, 1, 2]	HIT
8	3	[5, 1, 3]	FAULT (2 out)
9	1	[5, 3, 1]	HIT

Insert sequence: 1, 2, 3, 4, 5, 3
Replace sequence: $3 \to 4,\ 4 \to 5,\ 2 \to 3$
Hits = 3, Faults = 6, Hit ratio = 3/9 = 0.33, Fault ratio = 6/9 = 0.67

4. Comparative Analysis

4.1 Results

From the outputs we can see that MRU performed better than FIFO for the given input. MRU had 6 page faults while FIFO had 8, and MRU got 3 hits compared to only 1 hit in FIFO.

Metric	FIFO	MRU
Page Hits	1	3
Page Faults	8	6
Hit Ratio	0.11	0.33
Fault Ratio	0.89	0.67
Replacements	5	3

4.2 Step-by-step Comparison

		FIFO		MRU	
Step	Page	Frames	Result	Frames	Result
1	1	[1]	FAULT	[1]	FAULT
2	2	[1, 2]	FAULT	[1, 2]	FAULT
3	3	[1, 2, 3]	FAULT	[1, 2, 3]	FAULT
4	4	[2, 3, 4]	FAULT	[1, 2, 4]	FAULT
5	5	[3, 4, 5]	FAULT	[1, 2, 5]	FAULT
6	1	[4, 5, 1]	FAULT	[2, 5, 1]	HIT
7	2	[5, 1, 2]	FAULT	[5, 1, 2]	HIT
8	3	[1, 2, 3]	FAULT	[5, 1, 3]	FAULT
9	1	[1, 2, 3]	HIT	[5, 3, 1]	HIT

For the first three pages both algorithms behave the same way, because the frames are empty and pages are only being loaded. The difference starts from page 4. At this point FIFO removes page 1 as it is the oldest page in memory. On the other hand, MRU removes page 3 since it was the last page used.

After that, pages 1 and 2 are requested again at steps 6 and 7. In MRU they are still present in the frames, so both of them become hits. In FIFO, pages 1 and 2 were already removed at steps 4 and 5, so these references cause faults again. FIFO only gets a hit at the last step, when page 1 is requested while it is still in memory.

4.3 Why MRU Performs Better Here

The reference string 1, 2, 3, 4, 5, 1, 2, 3, 1 accesses five pages one after another and then goes back to page 1. Since there are only 3 frames, all the pages cannot stay in memory at the same time. In this type of looping pattern, the page that was loaded earliest is actually the one that will be needed next. FIFO removes exactly that page every time, which is why it keeps getting faults. MRU removes the page used most recently, and that

page is not needed again until much later in the string. As a result pages 1 and 2 stayed in memory and gave hits.

To check whether the result depends only on 3 frames, I ran both programs again with frame sizes from 1 to 5. The number of page faults are shown below.

Frames	FIFO faults	MRU faults
1	9	9
2	9	8
3	8	6
4	8	5
5	5	5

MRU gave fewer faults when the number of frames was 2, 3 and 4. With 1 frame both algorithms fault on every page. With 5 frames all the pages fit in memory, so both of them only have the 5 faults that happen when each page is loaded for the first time.

4.4 When FIFO Performs Better

Although MRU did better here, it is not always the better algorithm. To see the opposite case, both programs were also run on the following string with 3 frames, where pages 4, 5 and 6 are used repeatedly within a short time:

1, 2, 3, 4, 5, 4, 5, 6, 5, 6 (3 frames)

Metric	FIFO	MRU
Page Faults	6	10
Hit Ratio	0.40	0.00

In this case MRU got a fault on every reference. When page 5 comes in, MRU removes page 4 because it was used last, but page 4 is requested in the very next step. Then page 4 removes page 5, and the same thing keeps happening. FIFO instead removes the older pages 1, 2 and 3, so pages 4, 5 and 6 remain in memory and it gets 4 hits.

Most real programs show this kind of behaviour, which is called temporal locality. Variables inside a loop or the current function's stack are accessed again and again within a short time, and MRU performs poorly for such programs. It is more suitable for cases like a database scanning a large file repeatedly from start to end.

Another point is that FIFO may suffer from Belady's anomaly. This means increasing the number of frames can sometimes increase the number of page faults. MRU is a stack algorithm, so this anomaly does not occur in MRU.

4.5 Summary

Aspect	FIFO	MRU
Page removed	The one loaded earliest	The one used most recently
Effect of a hit	No change	Page moves to most-recent position
Good for	Programs with temporal locality	Repeating scans bigger than memory
Code complexity	A simple queue	Must update order on every hit

5. Conclusion

Here FIFO and MRU page replacement algorithms were implemented and compared using the reference string 1, 2, 3, 4, 5, 1, 2, 3, 1 with 3 frames. FIFO produced 8 page faults and 1 hit, while MRU produced 6 page faults and 3 hits, so MRU performed better for this input. This happened because the reference string repeats in a loop and MRU keeps the older pages in memory. However, for a string with temporal locality FIFO gave better results than MRU. So the better algorithm depends on the page access pattern of the program.

6. GitHub

Click here to view source code